initialized in a line or two of code, and they give you lazy evaluation for free (RSpec never runs the block until it's actually needed).

You've seen all there is to let. But we'd like to show you a little more about the other two techniques.

Hooks

Hooks are for situations where a let block just won't cut it. For instance, your shared setup code may need to have side effects such as modifying global configuration or writing to a file. Or you may be implementing cross-cutting concerns like database transactions.

Writing a hook involves two concepts. The *type* of hook controls *when* it runs relative to your examples. The *scope* controls *how often* your hook runs.

Let's look at both of these concepts in turn.

Type

There are three types of hooks in RSpec, named after when they run:

- before
- after
- around

The before and after hooks are related, so let's dive into those first.

before and after

As the name implies, your before hooks will run before your examples.

after hooks are *guaranteed* to run after your examples—even if the example fails or the before hook raises an exception. These hooks are intended to clean up after your setup logic and specs.

Here's an example that stashes the ENV hash containing your app's environment variables (so that your specs can safely modify them without affecting other specs), then restores the hash afterward:

07-structuring-code-examples/04/before_and_after_hooks_spec.rb

```
RSpec.describe MyApp::Configuration do
  before(:example) do
    @original_env = ENV.to_hash
  end

  after(:example) do
    ENV.replace(@original_env)
  end
end
```